Gravity Shooter

プログラム説明資料



日本工学院専門学校ゲームクリエイター科　4年制28卒予定
末廣 航一

GitHubリポジトリ

作品概要

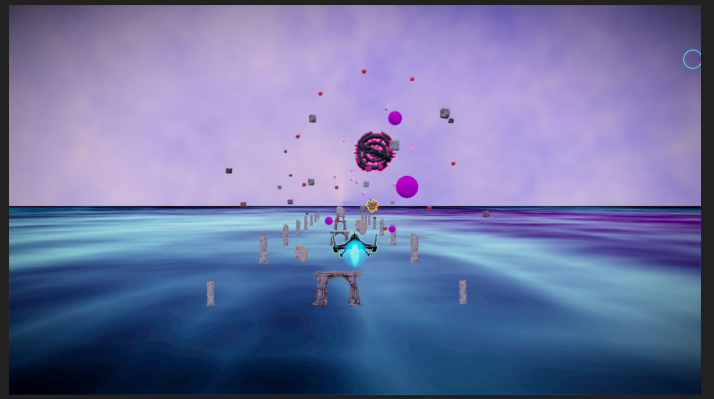


ジャンル : 3D奥スクロール・レールシューティング

制作時期 : 2026年5月～（制作途中 / 約5ヶ月）

開発環境・API :
- 言語 / IDE: C++20 / Visual Studio 2026
- 基盤技術: DirectX 12（Direct3D 12） / HLSL (Shader Model 6.6)
- サウンド: XAudio2
- OS / SDK: Windows 10・11 / Windows SDK 10.0.26100

外部ライブラリ :
- Assimp（3Dモデル読み込み）
- DirectXTex（テクスチャ・DDS読み込み）
- nlohmann json（敵ウェーブ・ステージ・設定データのパースと保存）
- Dear ImGui（インゲーム内製エディタおよびリアルタイムデバッグUI）



担当箇所 : プログラム全般（自作DirectX12ゲームエンジン設計、
ゲームプレイロジック、レンダリングパイプライン、内製ツール）
※効果音・BGMと、一部3Dモデル・UI素材はフリーおよびAI生成素材を使用

ゲームコンセプトと技術目標 :
- 『DEATH STRANDING』のような重厚な空気感と、『グラビティデイズ』のような「周囲の物を重力で引き寄せてぶつける」操作感を意識した3D重力シューティング。
- 最大1万オブジェクトに耐えうるスケーラブルな仮想エンティティ設計と、多重ポストプロセスが稼働する環境下でもCPU-GPU間の同期オーバーヘッドや不要な動的メモリ確保を排除し、**60FPS（1フレーム16.6ms以内）**で極めて安定して動作する堅牢なアーキテクチャの構築を目標に設定。

2D/3D重み付けスコアリングと透過を防ぐ射線遮蔽判定

【要件と課題】多数の敵機や浮遊ガレキが入り乱れる3D空間において、プレイヤーが意図した敵へ直感的に照準を吸着させつつ、ビル等の遮蔽物の裏にいる見えない敵への透過ロックオンや、遠方の敵への誤吸着を防止する必要があった。

【比較と選定理由】画面中央からの2D距離のみでは画面奥の遠い敵を誤認識し、3D距離のみでは画面端の敵を不自然に吸着してしまう。そこで**2D照準距離と3D深度距離を重み付け合算したスコアリング式**と、**Raycastによる射線遮蔽（Line of Sight）判定**を組み合わせたハイブリッド方式を採用した。

【実装の工夫と代償】

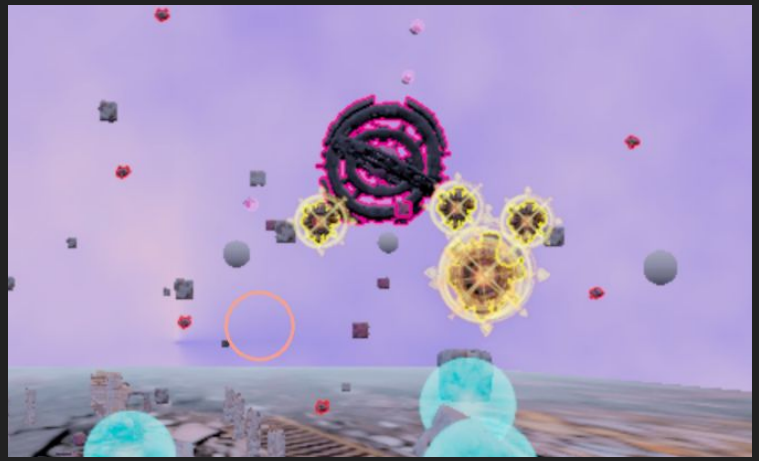
初回同期・以降非同期の遮蔽判定（Amortization）：初回検出時は即座に同期Raycastを実行して壁裏への一瞬の透過表示を防止。追尾継続中は0.1秒ごとの非同期Raycastに切り替えることで、毎フレームの衝突判定によるCPU負荷を分散させた。

重み付けスコア：`score = std::sqrt(dist2DSq) * weight2D_ + dist3D * weight3D_;` により、照準近傍かつ手前の敵を最優先でロックオン。

【定量的な成果】

遮蔽物が多いステージでも誤作動せず、プレイヤーの視線誘導に沿った自然で手応えのあるマルチロックオン操作を実現した。

```cpp
// PlayerTargetingComponent.cpp : 2D/3D重み付け走査と初回同期遮蔽判定
float score = std::sqrt(dist2DSq) * weight2D_ + dist3D * weight3D_;
if (score < bestScore) {
    if (!cache.hasCheckedOnce) {
        // 初回は同期Raycastで遮蔽を確定し、壁裏敵の一瞬の透過ロックを防止
        RaycastHit hitInfo{};
        bool hit = collisionManager->Raycast(ray, hitInfo, dist3D + 10.0f);
        cache.canSee = (!hit || hitInfo.distance >= dist3D - 1.0f);
        cache.hasCheckedOnce = true;
    } // 2回目以降は0.1秒間隔の非同期RaycastでCPU負荷を分散
}
```



戦術ステートマシンと自機移動ベクトルに基づく未来位置予測

【要件と課題】 単調な直線射撃の繰り返しを防ぎ、特攻機・定点機・狙撃機といった明確な役割分担を持つ立体的な戦闘を構築する必要があった。また、自機と敵機（ボス・随伴機）が秒速　15mでレール移動する奥スクロール空間において、通常の射撃では弾丸が自機に届かず後方へ不自然に置き去りにされる相対速度の弾道破綻が発生。

【比較と選定理由】 場当たり的な条件分岐を排し、役割ごとの振る舞い（特攻　/定点/狙撃）と行動フェーズ（接近 → 交戦 → 特攻/離脱）を分離した **階層型ステートマシン** を採用。さらに、静止座標系での射撃では照準が大きくズレるため、 **自機の移動ベクトルから到達時間を逆算する未来位置予測（Predictive Lead Shooting）**に加え、**発射母機のワールド前進速度を弾丸に加算する「ガリレイ慣性変換（ Galilean Invariance）」**を統合導入。

【実装の工夫と代償】

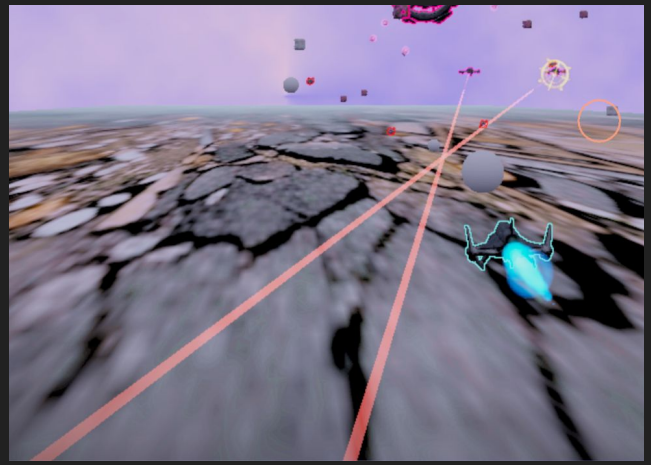
- **未来予測射撃と照準予告（ Telegraph）**: 弾速と距離から到達時間 **travelTime** を算出し、自機移動ベクトルを乗じた未来予測座標（ **predictedTarget = playerPos + playerVel * travelTime**）へ射撃。直前まで赤色レーザーで射線を予告し、プレイヤーに「見てから回避する」手応えを設計。
- **ガリレイ慣性補正のベクトル合成** ：高速移動する母機の前進速度 `parentVelocity` を弾丸初速にベクトル合成。母機の旋回姿勢（ Right/Up軸）と弾道基底を同期させ、高速公転やカーブ旋回時でも弾幕がスカスカにならず自機を正確に捕捉する物理整合性を確立。

【定量的な成果】

敵の役割分担が明確化し、直進し続けると狙撃され、静止すると特攻される緊張感ある戦闘を実現。高速移動するボス戦でも弾幕がプレイヤーを正確に狙い撃ち、立体的なドッグファイトを成立。

```cpp
// RailShooterEnemyComponent.cpp : 弾丸到達時間と未来予測座標の算出
float dist = std::sqrt(dx * dx + dy * dy + dz * dz);
float travelTime = std::clamp(dist / (std::max)(bulletSpeed_, 1.0f), 0.0f, 1.2f);

// 自機の現在位置と移動速度ベクトルから未来予測座標を逆算
Vector3 predictedTarget = { playerPos.x + playerVel.x * travelTime,
                            playerPos.y + playerVel.y * travelTime,
                            playerPos.z + playerVel.z * travelTime };
Vector3 aimDir = Normalize(predictedTarget - myPos);
```



仮想インスタンスによるデータ管理と初期化時プール事前確保

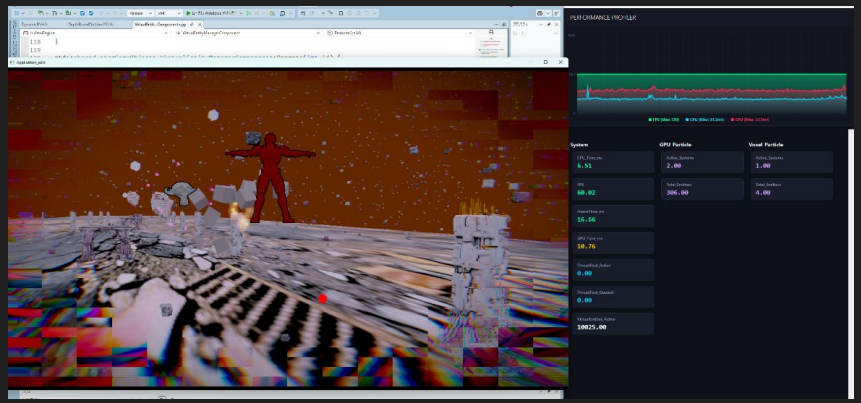
【要件と課題】最大1万個規模の負荷検証に耐えうる拡張性を備えつつ、実ゲーム内で数十～　100個規模のガレキが絶え間なく循環（吸引　➔ 防御 ➔ 投擲 ➔ 破砕）する環境において、すべてを完全な **GameObject** として生成・走査すると膨大なメモリ消費と毎フレームの走査で　CPUが処理限界を迎える。また、ゲーム進行中の頻繁な動的メモリ確保　（ **new / delete** ）は断片化と処理スパイクの原因となる。

【比較と選定理由】　全オブジェクトの常時実体化を廃止。通常時は座標・姿勢データのみを保持する「仮想インスタンス（ VirtualInstance）」として軽量管理し、プレイヤーの重力引き寄せや当たり判定が必要になった瞬間のみ、初期化時に事前確保したプールから　**実体化（ Promote）**させるハイブリッドライフサイクルを採用。

【実装の工夫と代償】

- **初期化時の事前確保（　ObjectPool）**：ゲームプレイ中に要求される最大実体数（数十個）分だけをシーン初期化時に一括事前確保（ Pre-allocation）し、インゲーム中の動的アロケーションをゼロ化。Handleと世代（Generation）管理により不正アクセスを防止。
- **動的昇格・降格（ Promote / Demote）**：重力干渉時に **Promote()** でプールから実体を貸し出し、敵への投擲や画面外離脱時には　**Demote()** でプールへ即座に返却して仮想データへ戻す疎結合な管理を構築。

```cpp
// VirtualEntityManagerComponent.h : 仮想データと事前確保プールの連携
struct VirtualInstance {
    int id;
    Vector3 position, rotation, scale;
    bool isPromoted = false;
    ObjectPool<GameObject>::Handle promotedHandle; // 事前確保プールへのHandle
};
// 重力干渉時のみ実体を割り当て、投擲・回収後は即座にプール返却
std::shared_ptr<GameObject> Promote(int id);
void Demote(int id);
```



【定量的な成果】

最大10,000オブジェクトの大量生成ベンチマーク（　CPU負荷0.066ms）に耐えうるスケーラビリティを実証。実ゲームプレイでは視認性とゲームテンポを最優先した最適密度（数十～　100個）のガレキ循環を、**動的メモリ確保ゼロ・常時　60FPS**で完全維持。

ExecuteIndirect による完全GPU駆動カリング

【要件と課題】 前作までにインスタンシング（Batch描画）は導入済みであり、ドローコール自体の削減は行えていた。しかし本作で大量のガレキや背景が密集するシーンを構築した際、**CPU側での視錐台カリング計算と、毎フレームの可視バッファ構築・転送処理**がCPUを圧迫。ゲームロジックの処理時間を圧迫し、フレームレート低下の原因となった。

【比較と選定理由】 CPU側でマルチスレッド化してカリングを行う手法も検討したが、CPU-GPU間のバッファ転送負荷や同期オーバーヘッドは解消できない。そこで、可視判定から描画引数の動的生成までをすべてGPU側で完結させる**ExecuteIndirectによる完全GPU駆動パイプライン**を自作・導入した。

【実装の工夫と代償】 GPUCulling.CS.hlsl 内で視錐台6平面との球判定を行い、可視オブジェクトのインデックスを`InterlockedAdd` でアトミックに加算して描画引数バッファを動的生成。境界部分のチラつきを防ぐため10%の保守的マージン（Conservative Culling）を設けた。

【定量的な成果】
CPUからの描画コマンド発行をExecuteIndirect の**1回**に集約。CPU側で行っていた可視判定ループとバッファ転送処理を排除し、描画前処理負荷を大幅に低減した。

```hlsl
// GPUCulling.CS.hlsl: 視錐台交差判定とGPU側での描画引数動的生成
if (isVisible) {
    uint visibleIndex = 0;
    // D3D12_DRAW_INDEXED_ARGUMENTS の InstanceCount をGPU側でアトミック加算
    InterlockedAdd(CommandBuffer[1], 1, visibleIndex);
    OutputInstances[visibleIndex] = outInst; // 可視インスタンスを行列バッファへ格納
}
```

宣言型 RenderGraph アーキテクチャとリソースバリア自動解決

【要件と課題】 空間歪み、Bloom、被写界深度（DoF）、輪郭抽出など十数個の描画パスが連携するパイプラインにおいて、DirectX 12のリソースバリア（状態遷移）を手動管理していたため、遷移指定の記述漏れや順序不整合によるGPUクラッシュのリスクに常に直面していた。

【比較と選定理由】 個別パスごとに手動でResourceBarrier をハードコードする手法は、描画順序の入れ替えや新規エフェクト追加のたびに全パスを修正する必要があり保守性が破綻する。そこで、パス間の依存関係からエンジンが実行順序と適切なバリア発行タイミングを自動解決する**RenderGraph（レンダリンググラフ）** を自作・導入。

【実装の工夫と代償】 各レンダーパスをクラスとしてカプセル化し、リソースの入出力を宣言型（Read / Write）で定義。エンジンが有向非巡回グラフ（DAG）のトポロジカルソートを行い、要求ステートと現在のステートを比較し**最小限の遷移バリアを一括生成・発行**する仕組みを構築。

【定量的な成果】
リソースバリアの不整合に起因するGPU警告やハングアップを完全に根絶。新規ポストエフェクトの追加や描画順序の変更を安全かつ即座に行える高い拡張性を確立。

```cpp
// RenderGraph.cpp : 状態追跡によるリソースバリアの自動計算と一括発行
for (const auto& usage : pass.GetResourceUsages()) {
    D3D12_RESOURCE_STATES currentState = resourceStates_[usage.resource];
    // 要求ステートと異なる場合のみ、最小限のバリアを自動生成
    if (currentState != usage.state) {
        barriers.push_back(CreateTransitionBarrier(usage.resource, currentState, usage.state));
        resourceStates_[usage.resource] = usage.state; // 状態キャッシュを更新
    }
}
commandList->ResourceBarrier(static_cast<UINT>(barriers.size()), barriers.data());
```

空間歪み・被写界深度と鮮明なHUD表示を両立する描画パス分離

【要件と課題】 重力場による激しい空間の歪み（屈折）や、被写界深度（DoF）・モーションブラーを画面全体に適用すると、プレイヤーの照準レティクルやHPゲージ、ロックオンマーカーまでが巻き込まれて歪み、ゲームプレイの視認性が著しく損なわれる問題が発生。

【比較と選定理由】 UIのテクスチャを別途クリアして全画面合成する手法はメモリ帯域を浪費する。そこで、自作RenderGraph上で描画パスを**「Pre-UIパス（3D世界のみに重力歪み・DoFを適用）」**と**「Post-UIパス（鮮明なUIを描画した後に最終トーンマップ）」**に論理レイヤー分離する設計を採用。

【実装の工夫と代償】 3Dシーンのレンダリング完了後、空間歪みと被写界深度のポストプロセスを先に適用してバックバッファへ解決。その後にスプライト・テキスト描画パスを実行し、歪みの影響を一切受けないクリーンな状態でHUDを重畳。

【定量的な成果】
重厚なサイバーパンク世界のビジュアル表現と、シューティングゲームとして最も重要な「照準や状況把握の快適さ」を高次元で両立。

```cpp
// RenderPipeline.cpp : Pre-UI と Post-UI のレイヤー分離構成
renderGraph.AddPass<MainOpaquePass>("MainOpaque");
renderGraph.AddPass<DistortionAndDoFPass>("PreUIPostProcess"); // 3D世界のみ歪み適用
renderGraph.AddPass<UIRenderPass>("InGameHUD");                // 鮮明なHUDを描画
renderGraph.AddPass<ColorGradingPass>("PostUIPass");           // 最終出力
renderGraph.Compile();
renderGraph.Execute(renderContext);
```

Descriptor Indexingによるバインドレス化と描画コマンド最適化

【要件と課題】 多種類の浮遊ガレキ（鉄骨、コンクリート、岩石片）や敵機を描画する際、オブジェクトごとに異なるテクスチャを割り当てる必要がある。しかし、描画ごとにディスクリプタテーブルを切り替える従来の方式ではCPU-GPU間のステート変更オーバーヘッドが多発し、インスタンシング描画の妨げとなっていた。

【比較と選定理由】 全テクスチャを1枚にまとめるテクスチャアトラス化は、解像度制限やUVマージン調整が必要となり制作上の制約が大きくなる。そこでDirectX 12 (Shader Model 6.6) の **Descriptor Indexing（Bindlessテクスチャ）** を導入し、シェーダー側でテクスチャを動的解決する手法を選択。

【実装の工夫と代償】 全テクスチャを単一のグローバルSRVディスクリプタヒープに集約し、ルートシグネチャに1度だけバインド。各オブジェクトからは定数バッファ経由で整数インデックス（uint textureID）のみを渡し、HLSL側で非有界配列 gTextures[textureID].Sample(...) として直接参照。

【定量的な成果】
異なるテクスチャを持つ無数のオブジェクトが混在しても、ディスクリプタテーブルのバインド切り替えコストがゼロになり、描画パイプラインの極めて高いスループットを達成。

```hlsl
// Bindless.hlsli: SM6.6非有界テクスチャ配列
Texture2D<float4> gTextures[] : register(t0, space1);

// Object3d.PS.hlsl: マテリアル番号による動的インデックス参照
float4 main(PixelInput input) : SV_TARGET {
    // オブジェクトごとの再バインド不要。IDから直接テクスチャをサンプリング
    float4 albedo = gTextures[gMaterial.textureIndex].Sample(gSampler, input.uv);
    return albedo * input.color;
}
```

非水密・薄型メッシュに対応する表面サンプリングフォールバック

【要件と課題】 前作ではビル等の単純な立体をボクセル化していたが、本作で多様な敵モデル（ドローンやスナイパー機）をボクセル化する際、**薄板ポリゴンや非水密（穴のあるオープン）メッシュにおいて従来の 3軸レイキャスト内外判定が失敗し、ボクセル生成数が0個になってしまう問題** に直面した。

【比較と選定理由】 アセット側をすべて完全な水密立体（閉じた形状）に制限するのは制作上の制約が大きすぎる。そこで、エンジン側で**「レイキャスト内外判定でボクセルが 0個になった場合、表面ポリゴン重心からグリッドへ直接サンプリングする表面フォールバック処理」**を導入し、あらゆるメッシュ形状を確実に破砕可能にする手法を採用した。

【実装の工夫と代償】

表面ポリゴン重心の直接サンプリング ：3軸レイキャスト内外判定でボクセル数が0個になった場合のみフォールバックを発動。各ポリゴンの重心座標（Centroid）を計算し、ボクセルグリッドの対応セルへ直接配置。

ビット結合ハッシュによる重複セルの排除 ：同一セル内に複数ポリゴンの重心が落ちた際の多重生成を防ぐため、グリッド座標（x, y, z）をビットシフト結合したキー（uint64_t）を用い、ハッシュセットで即座に重複をスキップして破片数を最適化。

【定量的な成果】

羽や装甲などの薄板ポリゴンや非水密メッシュを持つ敵モデルでも確実にボクセル化を成功させ、アセットの形状に左右されない堅牢な破砕処理を実現した。

```cpp
// ModelManager.cpp : 薄型・オープンメッシュに対する表面フォールバック
if (result.voxels.empty()) {
    for (const auto& mesh : model.meshes) {
        // 各ポリゴンの重心（Centroid）からグリッド座標へ直接サンプリング
        Vector3 centroid = (p0 + p1 + p2) / 3.0f;
        int gx = std::clamp((centroid.x - aabbMin.x) / voxelSize.x, 0, res.x - 1);
        ...
        result.voxels.push_back(newVoxel); // 確実にボクセル破片を生成
    }
}
```

